

Playing with Pinecone

1. Create Index

Explain clearly on all attributes of indexes

```
!pip install pinecone
from pinecone import Pinecone
from pinecone import ServerlessSpec
from google.colab import userdata
pc = Pinecone(api_key=userdata.get("PINECONE_API_KEY"))
index_name = "dvsgenaib1"

if not pc.has_index(index_name):
    pc.create_index(
        name=index_name,
        vector_type="dense",
        dimension=384,
        metric="cosine",
        spec=ServerlessSpec(
            cloud="aws",
            region="us-east-1"
        ),
        deletion_protection="disabled",
    )
```

2. List indexes

```
indexes = pc.list_indexes()
print(indexes)
```

3. Describe index

```
pc.describe_index(name="dvsgenaib1")
```

4. Upsert data

```
content = [
    "Coffee is a popular beverage made from roasted beans.",
    "Tea contains antioxidants and improves health.",
    "Laptop computers are used for programming and work.",
    "Vector databases store embeddings for similarity search.",
    "Neural networks are used in deep learning models.",
    "Espresso is a strong form of coffee.",
    "Green tea is known for its health benefits.",
    "Machine learning models require training and inference.",
    "Pinecone is used for storing vector embeddings.",
    "Cappuccino contains coffee, milk, and foam.",
    "Transformers are used in modern AI systems.",
    "Databases store structured and unstructured data.",
    "Black tea has caffeine and strong flavor.",
    "Embeddings convert text into numerical vectors.",
    "AI models use GPUs for faster computation.",
    "Latte is made with espresso and milk.",
    "Semantic search finds meaning in text.",
    "Coffee beans are grown in tropical regions.",
    "Vector similarity uses cosine similarity.",
    "AI helps automate many tasks."
]
```

syntax

```
from pinecone import Pinecone
pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index(host="INDEX_HOST")
index.upsert(
    vectors=[
        {
            "id": "A",
            "values": [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1],
            "metadata": {"genre": "comedy", "year": 2020}
        },
        {
            "id": "B",
            "values": [0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2],
            "metadata": {"genre": "documentary", "year": 2019}
        },
    ],
    namespace="example-namespace"
)
```

```
ids = [str(id) for id, values in enumerate(content)]
metadata = [{"text": values} for id, values in enumerate(content)]
ids, metadata
```

<https://huggingface.co/models?library=sentence-transformers&sort=downloads>
<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

```
!pip install sentence-transformers
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
```

```
embeddings = model.encode(content).tolist()
index = pc.Index("dvsgenaib1")

index.upsert(
    zip(ids, embeddings, metadata)
)
```

5. Data Retrieval - Semantic Search

```
query = "coffe is very strong"
q_embeddings = model.encode(query).tolist()
response = index.query(
    namespace="__default__",
    vector=q_embeddings,
    top_k=3,
    include_metadata=True,
    include_values=False
)
response
```

6. Delete Index

```
pc.delete_index("dvsgenaib1")
```

Final Verification

```
indexes = pc.list_indexes()
print(indexes)
```